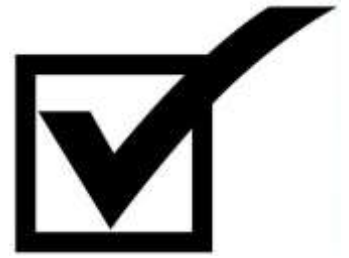




**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential



PYTHON PROGRAMMING

(355)

STATE 2023

APPLICATION KNOWLEDGE:

Test Score Statistics Reporter

_____ (330 points)

TOTAL POINTS

_____ (330 points)

Test Time: 90 minutes

How much do you pay for gas?

More and more people are moving to electric cars, but until then the rest of us pay for gasoline. While each gas pump might have different directions, you are going to find how much someone will pay at our pump.

Each patron will be asked if they want to use their discount. How the discount works at this pump is very simple. The customer can apply for a card, which has a 4-digit number on the face of the card. This number will determine what percent the customer could get at the pump. If your 4-digit number on your card is 1000 through and including 4000 you will get a 2% discount. If the number is 4001 through and including 7000 you will get a 3% discount. If the number is 7001 through and including 9999 you will get at 4% discount.

The customer will choose which level of octane they would like to purchase (Premium, Midgrade, or Regular). If they are using their discount, you will then find the discount price per gallon and apply it to the gallons the customer is purchasing.

Let's go find out how much someone is spending on gas today.

Requirements:

1. You must create a Python application named PP_355_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, PP_355_01_2345_6789.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. You will have two scripts. The first will be named info.py
4. Create a class named **Octane**
 - a. Create a constructor which accepts an argument (account name).
 - i. In the constructor, set the initial value of two attributes as follows:
 1. Name of account – provided through the constructor parameters.
 2. Current price – starts at the value of 0.
 - b. Create a method named **discount**
 - i. Which gives the discount to the price per gallon.
 - c. Create a method named **no_discount**
 - i. Which does not give a discount.
5. Create a function named **get_octanes**
 - a. Create three separate objects for the following accounts.
 - i. Premium 93, starting price: \$4.68
 - ii. Midgrade 89, starting price: \$4.55
 - iii. Regular 87, starting price: \$4.42
 - b. You will only be able to set the name and account number using the constructor.
 - i. Use the attribute to set the price.
 - c. Add each object to a list and return the list of account objects.
6. Create a function named **get_octane**
 - a. It accepts a list of account objects.
 - b. Display all of the accounts to the user displaying the name and price.
 - c. Prompts the user asking for which account to use.
 - d. Verifies and validates the user input using the list of accounts.
 - e. Returns the account object associated with the user's selection.
7. Create a script named gas_price.py.
 - a. Gets the list of accounts from the bank module.
 - b. Uses the get_discount function to determine the price per gallon after the discount based upon the card number.
 - i. Card numbers 1000 through and including 4000 gets a 2% discount.
 - ii. Card numbers 4001 through and including 7000 gets a 3% discount.
 - iii. Card numbers 7001 through and including 9999 gets a 4% discount.
 - c. You may use as my functions as you wish to accomplish the following:
 - d. Prompts the user if they would like to use their discount or to exit the application.
 - i. If they would like to use the discount
 1. Ask for the card number and validate that it is between 1000 and 9999.
 2. Give a list of the levels of octane and price before discount and ask which level they would like to use.
 3. Prints the level of octane and discount price

4. Asks how many gallons of gas are purchased
5. Finds a total cost using an object method.
- ii. If they choose not to use the discount.
 1. Give a list of the levels of octane and price before discount and ask which level they would like to use.
 2. Asks how many gallons of gas are purchased
 3. Finds a total cost using an object method.
- e. If the user's actions cause no change to the account balance, notify the user.
- f. If the option is invalid, prompt for user input again.
- g. The application needs to run until the user has opted to exit the program.

Example output

```
Would you like to use your discount?
(y)es
(n)o
e(x)it the application

y

What is your card number?

----List of levels of octane and price before discount.----
1-Premium 93      $  4.68
2-Midgrade 89     $  4.55
3-Regular 87      $  4.42
Which level of octane do you wish to use? 2
Midgrade 89 discount price: $4.37.

How many gallons of gas are you purchasing? 15
Your total cost of gas is: $65.52

Thank you; come again.
```

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 pts

The project is named according to the naming conventions _____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB _____ 10 pts

If the program does not execute, then the remaining items in this section receive a score of zero.

Application asks the user if they would like a discount. _____ 10 pts

Application allows user to exit the program. _____ 10 pts

Application asks the user for a card number if they ask for the discount. _____ 10 pts

Application lists the levels of octane and the price before discount and prints the new price. _____ 50 pts

Application asks the user which level of octane you would like to use. _____ 10 pts

Application asks the user how many gallons are purchased. _____ 10 pts

Application gives the total cost. _____ 20 pts

Source Code Review

Code is commented at the top, for each function, and as needed _____ 20 pts

Code uses reasonable and consistent variable naming conventions _____ 20 pts

Two scripts are present _____ 20 pts

The **Octane** class has a constructor with one argument and two methods which is constructed in a logical manner _____ 30 pts

The **get_octanes** function lists the 3 objects _____ 30 pts

The **get_octane** function is present and is constructed in a logical manner _____ 30 pts

The **get_discount** function is present and is constructed in a logical manner _____ 30 pts

Lists accounts from info module in gas_price module _____ 20 pts

Calculates the correct price. _____ 30 pts

Total Points: 420pts

Note to grader: This an example. They only are required to have 3 functions and 1 class, but they can have more functions if they choose. They do not have to use try and except. It is their choice. I did not add comments but the contestant is asked to add comments.

info.py

```
class Octane:
    def __init__(self, name: str):
        self.price = 0
        self.name = name

    def discount(self, discnt:float) -> None:
        self.price = format(self.price - (self.price * discnt),".2f")

    def no_discount(self) -> None:
        self.price = self.price

def get_octanes() -> list:
    level = []
    premium = Octane("Premium 93")
    premium.price = 4.68
    level.append(premium)
    midgrade= Octane("Midgrade 89")
    midgrade.price = 4.55
    level.append(midgrade)
    regular = Octane("Regular 87")
    regular.price = 4.42
    level.append(regular)
    return level

def get_octane(level: list) -> Octane:
    picked = -2 # start loop
    while picked < 0:
        ndx = 0
        if picked == -1: # had input error
            print(">>>Invalid choice - Please try again.<<<")
        print("\n----- List of levels of octane and price before discount.--
-----")
        for acc in level:
            ndx += 1
            print("{:>4}-{:<15} ${:>8.2f}" .format(ndx, acc.name, acc.price))
        try:
            picked = int(input("Which level of octane do you wish to use? "))
            if picked < 1 or picked > len(level):
                picked = -1 # input error
        except:
            picked = -1 # input error

        picked -= 1 # set account number to index
    return level[picked]
```

gas_price.py

```

import info

# This is one possible answers
def get_discount(start_price, card) -> float:
    if card >= 4000:
        price = start_price - start_price * 0.02
    elif card >= 7000 and card < 4000:
        price = start_price - start_price * 0.03
    else:
        price = start_price - start_price * 0.04
    return price

def print_error(text: str) -> None:
    print("#" * (len(text) + 8))
    print("### " + text + " ###")
    print("#" * (len(text) + 8))
    print()

def total_price(ending_price):
    gallon = int(input("How many gallons of gas are you purchasing? "))
    t_price = ending_price * gallon
    return t_price

gas_price = info.get_octanes()
keep_running = True
while keep_running:
    answer = input("Would you like to use your
discount?\n\t(y)es\n\t(n)o\n\t(e)xit the application.\n\t")
    answer = answer.lower()
    if answer == "y":
        cards = 0
        while cards == 0:
            card_num = int(input("What is your card number? "))
            if card_num > 1000 or card_num < 9999:
                cards = 1
            else:
                print("Invalid card number.")
        grade_to_use = info.get_octane(gas_price)
        starting_price = grade_to_use.price
        ending_price = get_discount(starting_price, card_num)
        print()
        print("-> {} discount price: ${:.2f}".format(grade_to_use.name,
ending_price))
        print()y
        keep_running = False

    elif answer == "n":
        grade_to_use = info.get_octane(gas_price)
        starting_price = grade_to_use.price
        ending_price = starting_price
        print()
        keep_running = False
    elif answer == "x":
        keep_running = False
    else:

```

```
        print_error("I did not understand.")

total = total_price(ending_price)
print("Your total cost of gas is: ${:.2f}".format(total))
print("Thank you; come again.")
```